```
### Team: Nicholas Chapman ###

.data
array: .space 10                # 10 8 bit numbers = 10 bytes
qoutients: .space 10            # 10 8 bit numbers = 10 bytes
remainders: .space 10           # 10 8 bit numbers = 10 bytes

.text
        ## initial vars
        la s0, qoutients        # qoutients
        la s1, remainders       # remainders
        li s2, 0x11000040       # 7 seg
        li s3, 0x11000020       # LEDs
        la s4, array            # array
        li t0, 0x11000000       # switch address into register file
        la t1, array            # load address of array
        li t2, 0                # load 0 into i
        li t3, 10               # load 10 into t3

### input stage
switch: ## loading thte 10 numbers from switches
        ### delay to enter switches
        li t5, 0                # i = 0
        li t6, 50000000         # n = 15 mil cycles (8 seconds)
        delay:                  # start the delay for input
        addi t5, t5, 1          # i++
        lbu a0, 0(t0)           # load from switches
        sw a0, 0(s2)            # display entered number on 7seg
        bne t5, t6, delay       # if t5 != t6 continue the delay

        ### end delay
        lbu t4, 0(t0)           # load a 8 bit number
        sb t4, 0(t1)            # switch data into array
        addi t1, t1, 1          # shift array pointer by 1 byte
        addi t2, t2, 1          # increment i++

        ### delay to show LED
        li t5, 0                # i = 0
        li t6, 20000000         # n = 20 mil cycles (1 seconds)
        delay2:                 # start the second delay
        addi t5, t5, 1          # i++
        sw a0, 0(s2)            # display entered number on 7seg
        sw t3, 0(s3)            # show 2 LEDs on to signify waiting period
        bne t5, t6, delay2      # if t5 != t6 continue the delay
        sw zero, 0(s3)          # when delay is over reset the LEDs to off
        ### end delay
        bne t2, t3, switch      # if i != 10, go to switch



### dividing stage
        la t0, array            # load array start address
        mv a0, s0               # load qoutients array start address
        mv a1, s1               # load remainders array start address
        li t5, 0                # set s to 0
        li t6, 10               # load 10 into t6
```

```
divide: ## dividing each number in the array by 3
        lbu t1, 0(t0)          # number from array into t1
        li t3, 0               # quotient = 0
        mv t4, t1              # copy t1 into t4 (working number)
        li t2, 3               # divisor 3
        blt t4, t2, divisiable # if the number < 3, don't do any subtraction
        subtract:
                sub t4, t4, t2  # working - 3
                addi t3, t3, 1  # quotient + 1
                bge t4, t2, subtract   # if working is greater than 3,
subtract again
        ## number has been divided, not greater than 0
        beqz t4, divisiable    # go to divisible if remainder = 0
        divisiable:            # if no remainder, skip here (perfectly
divisable)
        sb t3, 0(a0)           # quotient to quotient array
        sb t4, 0(a1)           # remainder to remainder array
        addi t0, t0, 1         # shift array pointer by 1 byte
        addi a0, a0, 1         # shift qoutients by 1 byte
        addi a1, a1, 1         # shift remainders by 1 byte
        addi t5, t5, 1         # s++
        li t6, 10              # t6 = 10
        bne t5, t6, divide     # if s!=10, repeat for next number


        ## call sort subroutine
        mv a0, s0              # qoutients address to argument
        call sort              # sort qoutients
        mv a0, s1              # remainders address to argument
        call sort              # call sort subroutine

        ## show sorted qoutients on 7 seg
        mv a0, s0              # qoutients address to argument
        mv a1, s1              # remainders address to argument
        li t0, 0               # i = 0
        li t1, 10              # 10
        loop:                  # loop start
                lb t2, 0(a0)    # put qoutient into t2
                lb t3, 0(a1)    # put remainder into t3
                sb t2, 0(s2)    # store qoutients into 7 seg
                sb t3, 0(s3)    # store remainders into LEDs
                addi t0, t0, 1  # i++
                add a0, s0, t0  # shift qoutient array to next number
                add a1, s1, t0  # shift remainder array to next number
                ### delay
                li t5, 0               # i = 0
                li t6, 25000000        # n = 20 mil cycles (1 seconds)
                delaytwo:              # start the delay between showing the
qoutient and remainder
                addi t5, t5, 1         # i++
                bne t5, t6, delaytwo   # if i != t6 loop again
                ### delay
                ble t0, t1, loop# if i < 10, loop
end:                                   # this is the end of the program
j end                                  # loop this end forever
```

```
### SUBROUTINE ###
### sorting stage
sort:
        mv t0, a0               # copy base address to t0
        li t4, 0                # j
        li t5, 0                # i
        jcycle:                 # the is the beginning of the inner loop
        lbu t1, 0(t0)           # Array[j]
        lbu t2, 1(t0)           # Array[j+1]
        ble t1, t2, noswap      # branch if Array[j] < Array[j+1]
                mv t3, t1       # Array[j] into temp (t3)
                sb t2, 0(t0)    # Array[j+1] into Array[j]
                sb t3, 1(t0)    # temp(old j) into Array[j+1]
        noswap:                 # if a swap is not needed, skip to here
        addi t4, t4, 1          # j++
        addi t0, t0, 1          # add 1 to old address
        li t6, 9                # 10 (N - 1)
        sub t6, t6, t5          # N - i - 1
        bgt t6, t4, jcycle      # if j < N - i - 1, go to check array[j] >
array[j+1]
        li t4, 0                # j = 0
        addi t5, t5, 1          # i++
        li t6, 9                # 10 (N - 1)
        mv t0, a0               # copy base address to t0
        blt t5, t6, jcycle      # jcycle until i = N - 1
        ret                     # return to call

### need to wait some time here ###
```